**INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY, BHUBANESWAR**
Department of Computer Science & Engineering | DAA Laboratory (CS3101)

# LAB 08 REPORT: DYNAMIC PROGRAMMING DESIGN PARADIGM

**Student:** Rishav Kumar **Roll:** B125099 **Branch:** B.Tech CSE (B1) **Instructor:** Dr. A. K. Dash **Date:** Oct 2026

| Q. No. | Problem | Algorithmic Paradigm | Time | Space | Status |
|---|---|---|---|---|---|
| Q1 | Minimum Coin Change | Unbounded Knapsack Tabulation DP | $\mathcal{O}(nV)$ | $\mathcal{O}(V)$ | Verified |
| Q2 | Coin Change Ways | Combinatorial Order-Free DP | $\mathcal{O}(nV)$ | $\mathcal{O}(V)$ | Verified |
| Q3 | Longest Common Subsequence | Prefix Tabulation & Direction DP | $\mathcal{O}(mn)$ | $\mathcal{O}(mn)$ | Verified |
| Q4 | Longest Increasing Subsequence | Subsequence Tabulation & Fast Search | $\mathcal{O}\frac{n^2}{\mathcal{O}}(n \log n)$ | $\mathcal{O}(n)$ | Verified |
| Q5 | Max Sum Increasing Subsequence | Weighted Subsequence DP | $\mathcal{O}(n^2)$ | $\mathcal{O}(n)$ | Verified |
| Q6 | Edit Distance with Traceback | Levenshtein Distance Matrix DP | $\mathcal{O}(mn)$ | $\mathcal{O}(mn)$ | Verified |
| Q7 | Rod Cutting with Reconstruction | Unbounded Substructure DP | $\mathcal{O}(n^2)$ | $\mathcal{O}(n)$ | Verified |
| Q8 | Optimal BST (OBST) | CLRS Tabulation Tree DP | $\mathcal{O}(n^3)$ | $\mathcal{O}(n^2)$ | Verified |
| Q9 | Collatz Conjecture | Modular Orbit Simulation + 64-bit Overflow | $\mathcal{O}(\text{orbit})$ | $\mathcal{O}(\text{path})$ | Verified |

## Q1: Minimum Coin Change Problem Unbounded Knapsack DP O(nV)

**Problem Statement (From Lab Sheet)**

**Given an integer array of coin denominations $C = \{c_1, c_2, ..., c_n\}$ representing coins of different values, and an integer target amount $V$, find the minimum number of coins needed to make up that amount. You may assume an infinite supply of each coin denomination. If that amount of money cannot be made up by any combination of the coins, return $-1$.**

**Mathematical Derivation & DP Recurrence:** Greedy approaches fail for non-canonical denomination systems (e.g., $C = \{1, 5, 6, 9\}$ and $V = 11$: greedy chooses $9 + 1 + 1 = 3$ coins, whereas optimal is $5 + 6 = 2$ coins). Let $\mathrm{dp}[v]$ denote the minimum coins needed for amount $v \in [0, V]$:

$$\mathrm{dp}[v] = \begin{cases} 0 & \text{if } v = 0 \\ \min_{c_j \le v}(\mathrm{dp}[v - c_j] + 1) & \text{if } v > 0 \\ \infty & \text{otherwise} \end{cases}$$

**Traceback & Memory Complexity:** An auxiliary array `choice[v]` records the exact coin denomination selected at each step, enabling full multiset reconstruction in $\mathcal{O}(V)$ time.

- **Time Complexity:** $\mathcal{O}(nV)$ states and transitions.
- **Space Complexity:** $\mathcal{O}(V)$ auxiliary memory footprint.

**Verification Trace:**

- Test 1 ($C = \{1, 5, 6, 9\}, V = 11$): **Min Coins: 2** $[5 + 6]$ (Optimal).
- Test 2 ($C = \{3, 5\}, V = 7$): **Status: –1** (Unreachable amount).
- Test 3 ($C = \{186, 419, 83, 408\}, V = 6249$): **20 coins** (Optimal).

**Complete Algorithmic Implementation ( `Q1/main.c` ):**

```
int solveMinCoins(int coins[], int n, int V,
                  int dp[], int choice[])
{
    dp[0] = 0;
    choice[0] = 0;
    for (int i = 1; i <= V; i++)
    {
        dp[i] = INT_MAX;
        choice[i] = -1;
    }
    for (int i = 1; i <= V; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (coins[j] <= i && dp[i - coins[j]] !=
INT_MAX)
            {
                if (dp[i - coins[j]] + 1 < dp[i])
                {
                    dp[i] = dp[i - coins[j]] + 1;
                    choice[i] = coins[j];
                }
            }
        }
    }
    return (dp[V] == INT_MAX) ? -1 : dp[V];
}
```

## Q2: Coin Change — Total Number of Ways Combinatorial Order-Free DP O(nV)

**Problem Statement (From Lab Sheet)**

**Given an array of distinct positive integers representing coin denominations $C = \{c_1, c_2, ..., c_n\}$ and a target amount $V$, find the total number of distinct combinations of coins that sum up to $V$. You may assume an infinite supply of each coin denomination. The order of coins does not matter (e.g., $1 + 2$ and $2 + 1$ are considered the same combination).**

**Mathematical Derivation & DP Recurrence:** To prevent permutation counting (order independence), coins are processed in the **outer loop** and amounts $v \in [c_i, V]$ in the **inner loop**. Let $\mathrm{dp}[v]$ store combinations summing to $v$:

$$\boxed{\mathrm{dp}[v] \leftarrow \mathrm{dp}[v] + \mathrm{dp}[v - c_i], \quad \mathrm{dp}[0] = 1}$$

**Order Independence & Numeric Safety:** Swapping loops would compute ordered sequences (permutations). To prevent integer overflow for large target values, accumulator is typed as `unsigned long long` (64-bit).

- **Time Complexity:** $\mathcal{O}(nV)$ iterations.
- **Space Complexity:** $\mathcal{O}(V)$ 1D rolling array.

**Verification Trace:**

- Test 1 ($C = \{1, 2, 5\}, V = 5$): **Total: 4 ways** ($[1 \times 5], [1 \times 3, 2], [1, 2 \times 2], [5]$).
- Test 2 ($C = \{2, 3, 5, 6\}, V = 10$): **Total: 5 ways** ($[2 \times 5], [2 \times 2, 3 \times 2], [2 \times 2, 6], [2, 3, 5], [5 \times 2]$).

**Complete Algorithmic Implementation ( `Q2/main.c` ):**

```c
typedef unsigned long long ull;

ull countCoinCombinations(int coins[], int n,
                          int V, ull dp[])
{
    dp[0] = 1ULL;
    for (int v = 1; v <= V; v++)
    {
        dp[v] = 0ULL;
    }
    for (int i = 0; i < n; i++)
    {
        int coin = coins[i];
        for (int v = coin; v <= V; v++)
        {
            dp[v] += dp[v - coin];
        }
    }
    return dp[V];
}
```

## Q3: Longest Common Subsequence (LCS) Prefix Tabulation & Traceback O(mn)

**Problem Statement (From Lab Sheet)**

**Given two sequences $X = \langle x_1, x_2, ..., x_m \rangle$ and $Y = \langle y_1, y_2, ..., y_n \rangle$, compute the length of their longest common subsequence and reconstruct the actual subsequence string. Write a program in C and derive complexity.**

**Mathematical Derivation & Recurrence:** Let $L[i, j]$ denote the LCS length of prefixes $X[1...i]$ and $Y[1...j]$:

$$\boxed{L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(L[i-1, j], L[i, j-1]) & \text{if } x_i \neq y_j \end{cases}}$$

**Direction Matrix & Reconstruction:** Matrix $b[i, j] \in \{\nwarrow, \uparrow, \leftarrow\}$ tracks optimal choices. Traceback starts at $(m, n)$ and walks backwards to index $(0, 0)$ in $\mathcal{O}(m + n)$ time.

- **Time Complexity:** $\mathcal{O}(mn)$ table construction.
- **Space Complexity:** $\mathcal{O}(mn)$ for $L$ and $b$ matrices.

**Verification Trace:**

- Test 1 (CLRS): $X = \mathrm{ABCBDAB}, Y = \mathrm{BDCABA} \rightarrow$ **Length: 4** | LCS: `"BCBA"`.
- Test 2 (DNA): $X = \mathrm{AGGTAB}, Y = \mathrm{GXTXAYB} \rightarrow$ **Length: 4** | LCS: `"GTAB"`.

**Complete Algorithmic Implementation ( `Q3/main.c` ):**

```c
int computeLCS(const char *X, const char *Y,
               int m, int n, int **L, Direction **b)
{
    for (int i = 0; i <= m; i++) L[i][0] = 0;
    for (int j = 0; j <= n; j++) L[0][j] = 0;
    for (int i = 1; i <= m; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (X[i-1] == Y[j-1])
            {
                L[i][j] = L[i-1][j-1] + 1;
                b[i][j] = DIR_DIAG;
            }
            else if (L[i-1][j] >= L[i][j-1])
            {
                L[i][j] = L[i-1][j];
                b[i][j] = DIR_UP;
            }
            else
            {
                L[i][j] = L[i][j-1];
                b[i][j] = DIR_LEFT;
            }
        }
    }
    return L[m][n];
}
```

## Q4: Longest Increasing Subsequence (LIS) Patience Sorting & DP O(n^2) / O(n log n)

**Problem Statement (From Lab Sheet)**

**Given an integer array $A = [a_0, a_1, ..., a_{n-1}]$, find the length of the longest subsequence such that all elements of the subsequence are strictly increasing. Write a program in C and derive complexity.**

**Mathematical Derivation & Dual Algorithms:** Let $\mathrm{dp}[i]$ denote length of the LIS ending strictly at index $i$:

$$\mathrm{dp}[i] = 1 + \max(\{0\} \cup \{\mathrm{dp}[j] \text{ mid } 0 \le j < i \text{ and } A[j] < A[i]\})$$

**Dual Algorithmic Strategy:**
1. **Tabulation ($\mathcal{O}(n^2)$):** Tracks `parent[i]` for full element recovery.
2. **Patience Sorting ($\mathcal{O}(n \log n)$):** Maintains smallest tails array via binary search (`bsearch_tails`).

- **Complexity:** Time: $\mathcal{O}(n^2)$ Tabulation / $\mathcal{O}(n \log n)$ Fast Search. Space: $\mathcal{O}(n)$.

**Tabular State Trace ($A = [10, 22, 9, 33, 21, 50, 41, 60, 80]$):**

| $i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| $A[i]$ | 10 | 22 | 9 | 33 | 21 | 50 | 41 | 60 | 80 |
| $\mathrm{dp}[i]$ | 1 | 2 | 1 | 3 | 2 | 4 | 4 | 5 | 6 |
| **parent** | - | 0 | - | 1 | 0 | 3 | 3 | 5 | 7 |

**Verification Trace:** Length = **6**, LIS: $[10, 22, 33, 50, 60, 80]$. Identical in both $\mathcal{O}(n^2)$ and $\mathcal{O}(n \log n)$!

**Complete Algorithmic Implementation ( `Q4/main.c` ):**

```c
int solveLIS_DP(int arr[], int n,
                int dp[], int parent[])
{
    for (int i = 0; i < n; i++)
    {
        dp[i] = 1; parent[i] = -1;
    }
    int max_len = 1, max_idx = 0;
    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (arr[j] < arr[i] && dp[j] + 1 > dp[i])
            {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
        if (dp[i] > max_len)
        {
            max_len = dp[i]; max_idx = i;
        }
    }
    return max_idx;
}
```

## Q5: Maximum Sum Increasing Subsequence (MSIS) Weighted Subsequence DP O(n^2)

**Problem Statement (From Lab Sheet)**

**Given an array of $n$ positive integers $A = [a_0, a_1, ..., a_{n-1}]$, find the maximum possible sum of a strictly increasing subsequence. Write a program in C and derive complexity.**

**Mathematical Derivation & Core Principle:** Let $\mathrm{msis}[i]$ denote maximum sum of increasing subsequence ending at index $i$:

$$\mathrm{msis}[i] = A[i] + \max(\{0\} \cup \{\mathrm{msis}[j] \text{ mid } 0 \le j < i \text{ and } A[j] < A[i]\})$$

**Crucial Theoretical Distinction from LIS:** In $A = [1, 101, 2, 3, 100, 4, 5]$:
- **LIS:** $[1, 2, 3, 4, 5]$ (length 5, sum = 15).
- **MSIS:** $[1, 2, 3, 100]$ (length 4, sum = **106**).

Hence, subsequence length optimization does **not** optimize subsequence sum!
- **Complexity:** Time: $\mathcal{O}(n^2)$. Space: $\mathcal{O}(n)$ auxiliary memory with `long long`.

**MSIS State Table ($A = [1, 101, 2, 3, 100, 4, 5]$):**

| $i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $A[i]$ | 1 | 101 | 2 | 3 | 100 | 4 | 5 |
| $\mathrm{msis}[i]$ | 1 | 102 | 3 | 6 | 106 | 10 | 15 |
| **parent** | - | 0 | 0 | 2 | 3 | 3 | 5 |

**Verification Trace:** Optimal Sum = **106**, Subsequence: $[1, 2, 3, 100]$.

**Complete Algorithmic Implementation ( `Q5/main.c` ):**

```c
typedef long long ll;

int solveMSIS(int arr[], int n,
              ll msis[], int parent[])
{
    for (int i = 0; i < n; i++)
    {
        msis[i] = arr[i]; parent[i] = -1;
    }
    int max_idx = 0; ll max_sum = msis[0];
    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (arr[j] < arr[i] && msis[j] + arr[i] >
msis[i])
            {
                msis[i] = msis[j] + arr[i];
                parent[i] = j;
            }
        }
        if (msis[i] > max_sum)
        {
            max_sum = msis[i]; max_idx = i;
        }
    }
    return max_idx;
}
```

## Q6: Edit Distance with Traceback Information Levenshtein Matrix DP O(mn)

**Problem Statement (From Lab Sheet)**

**Given two strings $A$ of length $m$ and $B$ of length $n$, compute the minimum number of operations (insertions, deletions, or substitutions) required to transform $A$ into $B$, and print the traceback result. Write a program in C and derive complexity.**

**Mathematical Derivation & Recurrence:** Let $D[i,j]$ be min operations to convert $A[0...i-1]$ into $B[0...j-1]$:

$$D[i,j] = \begin{cases} D[i-1,j-1] & \text{if } A[i-1] = B[j-1] \\ 1 + \min \begin{cases} D[i-1,j] & \text{(Delete)} \\ D[i,j-1] & \text{(Insert)} \\ D[i-1,j-1] & \text{(Substitute)} \end{cases} & \text{if } A[i-1] \neq B[j-1] \end{cases}$$

Base cases: $D[i,0] = i$ (all deletes), $D[0,j] = j$ (all inserts). **Reconstruction Alignment Script:** Traceback traverses from $(m,n)$ to $(0,0)$ decoding atomic edit steps.

- **Complexity:** Time: $\mathcal{O}(mn)$. Space: $\mathcal{O}(mn)$ matrix storage.

**Verification Trace & Alignment Script ($A = \text{kitten}, B = \text{sitting}$):**

- **Distance: 3** (Replace 'k'->'s', Replace 'e'->'i', Insert 'g').

```
Source A:  k  i  t  t  e  n  _
           S  |  |  |  S  |  I
Target B:  s  i  t  t  i  n  g
```

- Test 2 ($\text{sunday} \rightarrow \text{saturday}$): **Distance: 3** (Insert 'a', 't', Replace 'n'->'r').

**Complete Algorithmic Implementation ( `Q6/main.c` ):**

```c
int computeEditDistance(const char *A, const char *B,
                        int m, int n, int **D)
{
    for (int i = 0; i <= m; i++) D[i][0] = i;
    for (int j = 0; j <= n; j++) D[0][j] = j;
    for (int i = 1; i <= m; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (A[i-1] == B[j-1])
            {
                D[i][j] = D[i-1][j-1];
            }
            else
            {
                int del = D[i-1][j] + 1;
                int ins = D[i][j-1] + 1;
                int sub = D[i-1][j-1] + 1;
                int min_c = del < ins ? del : ins;
                D[i][j] = sub < min_c ? sub : min_c;
            }
        }
    }
    return D[m][n];
}
```

## Q7: Rod Cutting with Reconstruction Unbounded Substructure DP O(n^2)

**Problem Statement (From Lab Sheet)**

**Given a rod of length $n$ inches and an array of prices $P = [p_1, p_2, ..., p_n]$, determine: (i) The maximum revenue obtainable by cutting up the rod and selling the pieces; (ii) The exact lengths of the pieces that constitute the optimal decomposition (reconstruction). Cuts are integral and sum of piece lengths must equal $n$. Write a program in C and derive complexity.**

**Mathematical Derivation & Recurrence:** Let $r[j]$ denote maximum revenue obtainable for length $j \in [0,n]$:

$$r[j] = \max_{1 \le i \le j} (p_i + r[j-i]), \quad r[0] = 0$$

Array $s[j] = \text{argmax}_{1 \le i \le j}(p_i + r[j-i])$ stores the optimal first cut size. **Cut Reconstruction & Complexity:** While rod length $j > 0$, print piece $s[j]$ and decrement $j \leftarrow j - s[j]$.

- **Time Complexity:** $\sum_{j=1}^{n} j = \Theta(n^2)$ table construction.
- **Space Complexity:** $\mathcal{O}(n)$ for $r[...]$ and $s[...]$ tables.

**Verification Trace (CLRS 10-inch Benchmark):**

- Prices $P = [1, 5, 8, 9, 10, 17, 17, 20, 24, 30]$:
  - Length 4: **Revenue $10** | Pieces: $[2 \text{ in} + 2 \text{ in}]$.
  - Length 7: **Revenue $18** | Pieces: $[1 \text{ in} + 6 \text{ in}]$.
  - Length 10: **Revenue $30** | Pieces: $[10 \text{ in}]$ (Leave rod uncut).

**Complete Algorithmic Implementation ( `Q7/main.c` ):**

```c
void solveRodCutting(int n, int price[],
                     int r[], int s[])
{
    r[0] = 0; s[0] = 0;
    for (int j = 1; j <= n; j++)
    {
        int max_val = INT_MIN, best_cut = -1;
        for (int i = 1; i <= j; i++)
        {
            int cur = price[i] + r[j - i];
            if (cur > max_val)
            {
                max_val = cur;
                best_cut = i;
            }
        }
        r[j] = max_val;
        s[j] = best_cut;
    }
}
```

## Q8: Optimal Binary Search Trees (OBST) CLRS Tabulation Tree DP O(n^3)

**Problem Statement (From Lab Sheet)**

**Given a set of $n$ distinct sorted keys $K = \langle k_1, k_2, ..., k_n \rangle$ with search probabilities $p_1, ..., p_n$, and $n+1$ dummy keys $d_0, ..., d_n$ with search probabilities $q_0, ..., q_n$, find the minimum expected search cost of a binary search tree. Write a program in C and derive complexity.**

**Mathematical Derivation & Recurrence:** Expected search cost $E[\text{Cost}] = \sum_{i=1}^{n}(\text{depth}(k_i)+1)p_i + \sum_{i=0}^{n}(\text{depth}(d_i)+1)q_i$. Subtree weight $w[i,j] = w[i,j-1] + p_j + q_j$, with $w[i,i-1] = q_{i-1}$.

$$e[i,j] = \min_{i \le r \le j}(e[i,r-1] + e[r+1,j]) + w[i,j]$$

Base cases: $e[i,i-1] = q_{i-1}$. Root matrix $\text{root}[i,j] = r$ records chosen roots.

- **Time Complexity:** $\mathcal{O}(n^3)$ triple loops (can be $\mathcal{O}(n^2)$ via Knuth's rule).
- **Space Complexity:** $\mathcal{O}(n^2)$ for 2D tables $e$, $w$, root.

**Verification Trace (CLRS 15.5 Benchmark):**

- $n = 5, p = [0.15, 0.10, 0.05, 0.10, 0.20], q = [0.05, 0.10, 0.05, 0.05, 0.05, 0.10]$:
- **Optimal Expected Cost: 2.7500**.
- **Tree Structure:** $k_2$ is root; left child $k_1$ (leaves $d_0, d_1$); right child $k_5$; $k_4$ left child of $k_5$; $k_3$ left child of $k_4$.

**Complete Algorithmic Implementation ( `Q8/main.c` ):**

```
void optimalBST(int n, double p[], double q[],
                double **e, double **w, int **root)
{
    for (int i = 1; i <= n + 1; i++)
    {
        e[i][i - 1] = q[i - 1];
        w[i][i - 1] = q[i - 1];
    }
    for (int l = 1; l <= n; l++)
    {
        for (int i = 1; i <= n - l + 1; i++)
        {
            int j = i + l - 1;
            e[i][j] = DBL_MAX;
            w[i][j] = w[i][j - 1] + p[j] + q[j];
            for (int r = i; r <= j; r++)
            {
                double t = e[i][r - 1] + e[r + 1][j] +
w[i][j];
                if (t < e[i][j])
                {
                    e[i][j] = t; root[i][j] = r;
                }
            }
        }
    }
}
```

## Q9: Collatz Conjecture (3n + 1 Problem) Modular Orbit Simulation O(orbit)

**Problem Statement (From Lab Sheet)**

**The Collatz Conjecture defines a recurrence for positive integer $n$: $T(n) = \frac{n}{2}$ if $n$ is even, and $3n+1$ if $n$ is odd. The sequence repeatedly applies $T$ until $n = 1$. Write a modular C program to analyze the trajectory of a user-provided starting value $n \ge 1$ and across an interval $[a, b]$, handling dynamic memory allocation and 64-bit integer overflow.**

**Mathematical Derivation & Engineering Invariants:**

$$T(n) = \begin{cases} \frac{n}{2} & \text{if } n \equiv 0 \bmod 2 \\ 3n+1 & \text{if } n \equiv 1 \bmod 2 \end{cases}$$

**Critical Systems Architecture in C:**

1. **Integer Overflow Defense:** Condition $n > \frac{\text{UINT64_MAX} - 1}{3}$ detects overflow **before** arithmetic multiplication.
2. **Geometric Reallocation:** Trajectory buffer resizes geometrically ($2\times$) via `realloc()` in amortized $\mathcal{O}(1)$ time.
3. **Interval Analysis:** Tracks stopping time, peak values, and distributions.

- **Complexity:** Orbit-dependent stopping time; Space: $\mathcal{O}(\text{path})$ points.

**Verification Trace:**

- $n = 27$: Stopping time: **111 steps** | Peak: **9,232** | Ends at 1.
- Interval $[1, 20]$: Longest orbit: $n = 18$ (20 steps) | Peak: $n = 15$ (160).
- Interval $[900, 1000]$: Champion $n = 937$ (173 steps, peak 250,504).

**Complete Algorithmic Implementation ( `Q9/main.c` ):**

```
uint64_t getNextCollatz(uint64_t n, int *overflow_flag)
{
    if ((n & 1ULL) == 0ULL) return n >> 1ULL;
    if (n > (UINT64_MAX - 1ULL) / 3ULL)
    {
        *overflow_flag = 1;
        return 0ULL;
    }
    return 3ULL * n + 1ULL;
}

Trajectory simulateTrajectory(uint64_t start_val)
{
    Trajectory traj = initTrajectory(start_val);
    uint64_t curr = start_val; int ov = 0;
    while (curr > 1ULL)
    {
        uint64_t next = getNextCollatz(curr, &ov);
        if (ov) { traj.overflow_occurred = 1; break; }
        appendTrajectoryPoint(&traj, next);
        curr = next;
    }
    return traj;
}
```

## Algorithmic Synthesis & Concluding Remarks

### Cross-Paradigm Comparison of Lab 08 Problems

| Q# | Problem Name | Key Theoretical Invariant | Core Data Structure | Efficiency |
|---|---|---|---|---|
| Q1 | Minimum Coin Change | Optimal substructure over amount $v$ | 1D Table + Choice Array | $\mathcal{O}(nV)$ \| $\mathcal{O}(V)$ |
| Q2 | Coin Change Ways | Combinatorial order independence (outer coin loop) | 1D Accumulator ( `ull` ) | $\mathcal{O}(nV)$ \| $\mathcal{O}(V)$ |
| Q3 | Longest Common Subseq. | Prefix optimality & character matching | 2D Table $L[m][n] + b[m][n]$ | $\mathcal{O}(mn)$ \| $\mathcal{O}(mn)$ |
| Q4 | Longest Increasing Subseq. | Monotonic tail endpoints / predecessor links | DP Array + `parent` / Tails | $\mathcal{O}\frac{n^2}{\mathcal{O}}(n \log n)$ |
| Q5 | Max Sum Increasing Subseq. | Sum accumulation invariant over strictly smaller elements | DP Array + `parent` ( `ll` ) | $\mathcal{O}(n^2)$ \| $\mathcal{O}(n)$ |
| Q6 | Edit Distance (Levenshtein) | 3-way atomic edit decomposition (Ins/Del/Sub) | 2D Cost Matrix $D[m+1][n+1]$ | $\mathcal{O}(mn)$ \| $\mathcal{O}(mn)$ |
| Q7 | Rod Cutting Reconstruction | Unbounded optimal cut substructure | Revenue $r[n]$ + Cut $s[n]$ | $\Theta(n^2)$ \| $\mathcal{O}(n)$ |
| Q8 | Optimal BST (OBST) | Subtree weight sum + root choice interval DP | 2D Tables $e[n+2][n+1], \text{root}$ | $\mathcal{O}(n^3)$ \| $\mathcal{O}(n^2)$ |
| Q9 | Collatz Conjecture | Arithmetic parity dynamics & overflow prevention | Dynamic Resizing Array | $\mathcal{O}(\text{orbit})$ \| $\mathcal{O}(\text{path})$ |

### Pedagogical & Algorithmic Takeaways

1. **Optimal Substructure & Overlapping Subproblems:** All problems (Q1–Q8) obey Bellman's Principle of Optimality: an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal sequence with regard to the state resulting from the first decision.
2. **Combinations vs. Permutations in DP (Q1 vs. Q2):** Loop ordering fundamentally dictates the combinatorial nature of DP state spaces. Placing coins in the outer loop enforces strict order invariance (combinations), while placing target amount in the outer loop counts permutations.
3. **Space Optimization Techniques (1D Rolling Arrays):** Problems exhibiting single-step dependence (Q1, Q2, Q7) collapse 2D state spaces into $\mathcal{O}(V)$ or $\mathcal{O}(n)$ memory without sacrificing correctness.
4. **Traceback & Certificate Reconstruction:** Computing optimal objective scalar values (e.g. lengths, costs) is insufficient in engineering applications; back-pointers ( `choice` , `parent` , `root` , `s` , direction matrix `b` ) recover the exact optimal certificate without increasing asymptotic time complexity.
5. **Subsequence vs. Substring Dynamics:** Subsequences (Q3, Q4, Q5) allow arbitrary non-contiguous element omission, requiring quadratic search over previous states, unlike contiguous substring algorithms which maintain local sliding windows.
6. **Systems Engineering & Overflow Defense in Number Theory (Q9):** When dealing with non-convergent or explosive recurrence relations like the $3n+1$ problem, dynamic geometric array resizing and pre-computation arithmetic guard bounds ($n > \frac{\text{UINT64_MAX}-1}{3}$) are imperative for numerical integrity.

### Theoretical vs. Empirical Verification Summary

| Q# | Test Instance | Theoretical / Benchmark Target | Program Output | Status |
|---|---|---|---|---|
| Q1 | $C = \{1, 5, 6, 9\}, V = 11$ | Optimal coins = 2 $(5 + 6)$ | 2 coins $[5, 6]$ | PASS |
| Q2 | $C = \{1, 2, 5\}, V = 5$ | Total combinations = 4 | 4 combinations | PASS |
| Q3 | CLRS $X = \text{ABCBDAB}, Y = \text{BDCABA}$ | LCS Length = 4, string "BCBA" | Length 4, string "BCBA" | PASS |
| Q4 | $A = [10, 22, 9, 33, 21, 50, 41, 60, 80]$ | LIS Length = 6 ($[10, 22, 33, 50, 60, 80]$) | Length 6 (both $O(n^2)$ & $O(n \log n)$) | PASS |
| Q5 | $A = [1, 101, 2, 3, 100, 4, 5]$ | Max Sum = 106 ($[1, 2, 3, 100]$) | Sum 106, 4 elements | PASS |
| Q6 | $A = \text{kitten}, B = \text{sitting}$ | Levenshtein Distance = 3 | Distance 3, aligned script | PASS |
| Q7 | CLRS 10-inch Benchmark, $n = 4, 7, 10$ | Revenues: \$10, \$18, \$30 | Exact revenue & cut pieces | PASS |
| Q8 | CLRS 15.5 OBST, $n = 5$ keys | Optimal Cost $e[1, 5] = 2.7500$ | Cost 2.7500, exact tree | PASS |
| Q9 | Collatz $n = 27$ & intervals | Stopping time 111, peak 9,232 | 111 steps, peak 9,232 | PASS |

**STUDENT CERTIFICATION:**

I hereby certify that all mathematical proofs, dynamic programming formulations, C implementations, and empirical traces in this Lab 08 report are my original work.

**Student Signature:** *Rishav Kumar* (Roll: B125099, CSE-B1)

**FACULTY EVALUATION:**

Verified algorithm correctness, dynamic programming substructure derivations, and execution traces for Lab 08.

**Instructor Signature:** *Dr. Ajaya Kumar Dash*